LingTu 系统手册

四足机器人自主导航全栈 — 架构 / 算法 / 运维 / 外场操作

- LingTu 系统手册
  - 第一部分 · 系统全景
    - 1. LingTu 是什么
    - 2. 架构总览(6 层)
    - 3. 核心概念
  - 第二部分 · 感知与定位
    - 4. 传感器
    - 5. SLAM / 定位
    - 6. 地图层
  - 第三部分 · 规划与控制
    - 7. 全局规划 PCT
    - 8. 局部规划 — CMU 预采样
    - 9. 路径跟踪 — Pure Pursuit
    - 10. CmdVelMux 优先级仲裁
    - WaypointTracker + Mission FSM
  - 第四部分 · 语义与智能
    - 11. SemanticPlannerModule — 5 级 Goal Resolver
    - 12. VisualServoModule — 视觉伺服 + 跟人
    - 13. 记忆层
    - 14. Agent Loop — LLM 多步
  - 第五部分 · 安全与接口
    - 15. Safety 层
    - 16. Gateway / Dashboard / MCP
    - 17. Teleop + 录 bag
    - 视频流方案对比
  - 第六部分 · 运维与外场
    - 18. S100P 部署
    - 19. 外场操作
    - 20. 故障处理速查表
    - 21. 标定
  - 第七部分 · 附录
    - 22. 性能指标(S100P aarch64)
    - 23. 已知限制 / 路线图

24. CLI / HTTP API / FAQ
致谢

LingTu 系统手册

> **对象**:想完整理解 LingTu 的工程师 + 想在外场驾驶它的操作员。 **阅读建议**:第一次先过第 1-3 节(全景)+ 第 19 节(外场操作),其他按需查。 **版本**:v1.8,2026-04-19。

第一部分 · 系统全景

1. LingTu 是什么

LingTu(灵途)是**四足机器人在户外 / 越野环境下的自主导航系统。**

产品定位: - 不是 SLAM 库、不是规划器库 — 是**从传感器到电机的完整栈** - 不是遥控工具 — 是**自主决策系统**(给目标,机器人自己走过去 + 避障 + 恢复) - 不是仿真玩具 — 是**生产级,跑在真实硬件上**(S100P / 地瓜机器人 RDK X5)

目标硬件: - **计算**:S100P(Nash BPU 128 TOPS,aarch64,Ubuntu 22.04 + ROS2 Humble) - **LiDAR**:Livox MID-360(10 Hz,360° × 59°) - **相机**:Orbbec Gemini 335(深度 + RGB) - **IMU**:Livox 内置 + IMU 融合 - **GNSS**:WTRTK-980 RTK(可选,外场加强) - **运动控制**:brainstem 四足控制板(Dart + gRPC)

设计理念: 1. **Module-First**:Module 是唯一运行时单元,Blueprint 是唯一组装方式 2. **组合而非继承**:9 个 stack 工厂函数组合出完整系统 3. **插件化后端**:算法层面零 if/else, `@register("category","name")` 切换实现 4. **C++ 做重活**:Python 负责编排,C++ 负责算法热路径(nanobind 桥接) 5. **raw DDS 而非 rclpy**:生产不依赖 ROS2 Python 运行时

2. 架构总览(6 层)

```
L6  Interface      Gateway(HTTP/WS/SSE) · MCP Server · Teleop
──────────────────────────────────────────────────────────────
L5  Planning       NavigationModule · GlobalPlannerService · WaypointTracker
──────────────────────────────────────────────────────────────
L4  Decision       SemanticPlanner · LLM · VisualServo · AgentLoop
──────────────────────────────────────────────────────────────
L3  Perception     Detector(BPU) · Encoder(CLIP) · Reconstruction
```

```
                    SemanticMapper · Episodic · Tagged · VectorMemory
──────────────────────────────────────────────────────────────────────
L2  Maps            OccupancyGrid · ESDF · ElevationMap · Terrain
                    LocalPlanner · PathFollower · TraversabilityCost
──────────────────────────────────────────────────────────────────────
L1  Hardware        ThunderDriver · CameraBridge · SLAM(Fast-LIO2/Localizer)
──────────────────────────────────────────────────────────────────────
L0  Safety          SafetyRing · GeofenceManager · CmdVelMux
```

规则:高层可以依赖低层,**低层不能依赖高层**。跨层通信通过 Module 端口(Port)而非直接 import。

3. 核心概念

概念	含义	代码入口
Module	运行时单元,持有 In/Out 端口 + lifecycle (setup/start/stop)	`core.module.Module`
Port	带类型 + 背压策略的消息通道	`core.stream.In[T]` / `Out[T]`
Blueprint	组装 Module 的 DSL,支持 autoconnect / 显式 wire / 多种 transport	`core.blueprint.Blueprint`
Stack	预组装好的一组相关 Module(driver / slam / maps 等)	`core.blueprints.stacks.*`
Registry	后端插件系统, `@register("slam","fastlio2")`	`core.registry`
NativeModule	C++ 子进程包装,watchdog + SIGTERM	`core.native_module`
Profile	启动预设,CLI `lingtu.py <profile>`	`cli/profiles_data.py`
Session	全系统状态机,idle / mapping / navigating	Gateway 单一真源

背压策略(Out port 可选): | 策略 | 行为 | 用途 | |—|—|—| | `all` | 全量广播,不丢 | 默认 | | `latest` | 只保留最新,丢旧 | 高频传感器 | | `throttle(dt)` | 最大速率 | 降频 | | `sample(n)` | 每 n 帧取 1 | 降采样 | | `buffer(size)` | 批量积攒 | 批处理 |

Profile 清单(`cli/profiles_data.py`): | Profile | 用途 | |—|—| | `stub` | 框架测试,无硬件 | | `dev` | 语义开发,stub driver | | `map` | 建图模式(fastlio2 + pgo) | | `nav` | **生产导航**,slam_profile=bridge | | `explore` | 自主探索(frontier + PCT) | | `tare_explore` | CMU TARE 分层探索 | | `sim` | MuJoCo 全栈仿真 | | `sim_nav` | 纯 Python 导航仿真,无 ROS2 |

第二部分 · 感知与定位

4. 传感器

设备	话题	频率	订阅方
Livox MID-360 LiDAR	`/nav/lidar_scan` , `/nav/imu`	10 Hz / 200 Hz	Fast-LIO2
Orbbec Gemini 335	`/camera/color/image_raw` , `/camera/depth/image_raw` , `/camera/camera_info`	~7 Hz (默认) / 30 Hz (可调)	CameraBridge
GNSS WTRTK-980	`/gnss/fix` (串口 NMEA → DDS)	10 Hz	GnssModule
IMU	`/nav/imu` (Livox 内置)	200 Hz	Fast-LIO2

CameraBridge 两套订阅后端(`src/drivers/thunder/camera_bridge_module.py`): - **DDS-first**(raw cyclonedds):无 ROS2 env 也能跑,是生产路径 - **rclpy fallback**:装了 ROS2 Python 运行时时用,带 watchdog + USB 重置 + L1/L2/L3 自恢复

相机旋转: `config/robot_config.yaml:camera.rotate=270` ,因为相机在机身上竖装。CameraBridge 在 `_on_ros2_color` 里 apply `cv2.rotate` ,下游拿到的已经是”正向”帧。

5. SLAM / 定位

两种 SLAM 模式(由 profile 决定):

模式	slam_profile	作用	使用场景
建图	`fastlio2`	Fast-LIO2 从 0 累积地图	`lt session map`

模式	slam_profile	作用	使用场景
定位	`localizer`	Fast-LIO2(里程计)+ ICP Localizer(对齐 saved map)	`lt session nav`
桥接	`bridge`	订阅外部 systemd `slam.service` 提供的话题	**nav profile 默认**
关闭	`none`	不跑 SLAM	stub/dev

TF 链:

```
map  ──(Localizer ICP)──→  odom  ──(Fast-LIO2)──→  body  ──(静态标定)──→
lidar/camera
```

- Localizer 发 `map → odom` TF,ICP 收敛时才发
- Fast-LIO2 发 `odom → body`,里程计实时
- `base_link → sensors` 从 calibration 结果静态发

SlamBridgeModule(本 session 新增): - 订阅 `/nav/odometry`, `/nav/map_cloud`, `/nav/saved_map_cloud`, `/localization_quality`, `/tf` - 抽 map→odom 变换通过 `map_odom_tf: Out[dict]` 发给 Gateway - Gateway 对 map_cloud 点云应用变换后再发 SSE → 浏览器看到 map_cloud 与 saved_map 对齐

漂移守卫: - Fast-LIO2 长期静置(>1h)IEKF 协方差爆炸,odometry 输出 (-500000, 200000) 之类 - slam_bridge 检测位置突变(>500m)或速度超限(>5 m/s) - 连续 5 帧坏 → 停止转发 odometry + 尝试触发 `sudo systemctl restart slam` - 这是**应急补丁**,根因(C++ IEKF) 待修

localization_quality 语义: - 浮点数,**低 = 好**(0.0 完美,0.1-0.3 正常,>0.3 警告,-1.0 ICP 失败)

6. 地图层

四种地图**共存**,各司其职:

地图	数据结构	用途	产生者
PCD	三维点云(pcl)	SLAM 原始输出 + Localizer 参考	Fast-LIO2 `map save`

地图	数据结构	用途	产生者
OccupancyGrid	2D uint8 网格	2D A* / RVIZ 显示	`OccupancyGridModule` 从点云投影
ElevationMap	2D float 网格	地形高度 / 坡度分析	`ElevationMapModule`
ESDF	2D 欧式有符号距离场	LocalPlanner 避障打分	`ESDFModule`
Tomogram	分层(slice)多层高度图	**PCT 全局规划输入**	`_build_tomogram` 从 PCD 构建

Tomogram 构建时机: 建图后调 `lt map save <name>` 触发完整 pipeline: 1. `/nav/save_map` → `map.pcd` + poses.txt 2. `_build_tomogram(pcd)` → `tomogram.pickle` 3. `_build_occupancy_snapshot` → `occupancy.npz` + `map.pgm` + `map.yaml`

存放: `~/data/inovxio/data/maps/<map_name>/`

TraversabilityCostModule(L2 融合): - 输入:OccupancyGrid + ElevationMap + ESDF + Terrain - 输出: `fused_cost` 给 NavigationModule, `slope_grid` 给 Gateway 显示

第三部分 · 规划与控制

7. 全局规划 PCT

PCT_Planner 来源:HKU/HKUST 开源(GPLv2)。论文:[Efficient Trajectory Planning for Off-Road]。

产物: - `ele_planner.so` — 3D A*(hex-grid,多 slice) - `traj_opt.so` — GPMP 轨迹优化(最小 jerk) - `a_star.so` - `libele_planner_lib.so`

只有 aarch64 binary,x86 开发机自动 fallback `_AStarBackend` (纯 Python 2D A*)。

算法步骤(`planner_wrapper.py: TomogramPlanner.plan()`): 1. world → grid index(`pos2idx`) 2. 高度 → slice index(`pos2slice`)← 处理楼梯 / 多层 3. C++ ele_planner.so 做 3D A*(带 traversability + slice 切换代价) 4. C++ traj_opt.so 做 GPMP(高斯过程运动规划,平滑连续曲线) 5. grid → world

输出: `np.ndarray (N, 3)` ,含 z。

性能(S100P, 30m 场地):80-300 ms / plan。

Goal Safety BFS(`global_planner_service.py`):用户点的 goal 可能在障碍里,BFS 挪到最近可通行 cell 再传给 PCT。不保证一定可解,失败则 mission_status=FAILED。

8. 局部规划 — CMU 预采样

来源:CMU `base_autonomy` ,TARE 原班人马。 **核心**: `src/nav/core/include/nav_core/local_planner_core.hpp` 。

思路:离线生成 ~1000 条候选路径,在线快速打分选最优。**不做在线 MPC 优化。**

候选路径库: - `src/base_autonomy/local_planner/paths/*.ply` — MATLAB 生成 - 7 组,每组 ~150 条,长度 1-2m 的弧线

在线(10 Hz): 1. 按 robot yaw 把路径库旋转 36 方向(OpenMP 并行) 2. 每条 voxel 对 terrain_map 检查碰撞 → 标记 blocked 3. `scorePath()` 打分: `score = (1 - √√(方向误差)) × 旋转代价² × terrain可通行权重` 4. 组间聚合 → 最优 group → 组内选单条 5. 发 `local_path` (20-30 点)

加速: | 技术 | 收益 | |—|—| | SoA + CSR 稀疏 | cache 友好 | | `scorePathFast` LUT(pow025 查表) | **2.08x** | | OpenMP 并行 36 方向 | aarch64 四核 | | xsimd NEON / AVX | 批量旋转 | | LTO + `-ffast-math` | 跨函数内联 |

耗时:3-8 ms / frame。

9. 路径跟踪 — Pure Pursuit

来源:CMU base_autonomy,适配四足参数。 **核心**: `src/nav/core/include/nav_core/path_follower_core.hpp` 。

```
L = clamp(v × k_lookahead, L_min, L_max)      # 自适应 lookahead
P = find_lookahead_point(local_path, L)
α = heading_to_P - robot_yaw
κ = 2 sin(α) / L                              # Pure Pursuit 曲率
cmd.linear.x  = v_target                      # 0.3-0.8 m/s 保守值
cmd.angular.z = κ × v_target
```

四足适配: - `no_rot_at_goal=true` :到终点不原地转 - 加速度限幅:避免 v_target 跃变导致抖动

耗时:<1 ms。

10. CmdVelMux 优先级仲裁

所有 cmd_vel 源**都不直接**发 driver,经过 `CmdVelMux`:

源	优先级	超时
Teleop(摇杆 / WebSocket)	**100**	0.5 s
VisualServo(视觉伺服)	80	0.5 s
NavigationModule recovery(倒车)	60	0.5 s
PathFollowerModule(巡航)	40	0.5 s

无活跃源时输出零速。操作员摇杆一动,PathFollower 自动让位。

WaypointTracker + Mission FSM

NavigationModule 用 WaypointTracker 把长 global_path 拆成连续 waypoints,每帧推送当前目标给 LocalPlanner。

Mission 状态(`lt status` 可见):

```
IDLE ──goto──→ PLANNING ──ok──→ EXECUTING ──arrived──→ ARRIVED
  ↑              |                |                     |
  |            fail              stuck                  |
  └── cancel ── FAILED ←─── RECOVERING ──can't recover──┘
```

卡住检测:2s 位移 <0.1m → 触发 recovery(倒车 0.3s + replan)。连续 3 次失败 → FAILED。

第四部分 · 语义与智能

11. SemanticPlannerModule — 5 级 Goal Resolver

用户说”去厨房”或”找红色椅子”,怎么变成 `goal_pose`?

5 级链路(`src/semantic/planner/.../goal_resolver.py`):

```
1. Tag Lookup        精确/模糊匹配 TaggedLocationStore    → goal_pose
2. Fast Path         场景图关键字 + CLIP 匹配(<200ms)   → goal_pose
3. Vector Memory     CLIP embedding + ChromaDB 检索       → goal_pose
```

```
4. Frontier          拓扑图信息增益探索                  → goal_pose
5. Visual Servo      VLM bbox 检测 + PD 跟踪             → goal_pose/cmd_vel
```

Fast-Slow 双进程: - **Fast Path**(System 1, ~0.17 ms):场景图关键字 + CLIP 分数 + 空间推理,融合权重 `label 35% + CLIP 35% + detector 15% + spatial 15%`,阈值 0.75 - **Slow Path**(System 2, ~2 s):LLM 推理 + ESCA 选择性 grounding(200 object → 15 object,92.5% token 削减) - **AdaNav 熵触发**:候选分数熵 >1.5 且 confidence <0.85 → 强制升级到 Slow Path

12. VisualServoModule — 视觉伺服 + 跟人

双通道输出,按距离切换: - **远(>3m)**:发 `goal_pose` → NavigationModule 正常规划 - **近(<3m)**:发 `cmd_vel` → CmdVelMux(优先级 80)→ 绕过规划,直接 PD 跟踪

组件: - `BBoxNavigator`:bbox + 深度 → 3D 位置 → PD 控制 - `PersonTracker`:VLM 选人 + CLIP Re-ID 防丢失 - `vlm_bbox_query`:Open-vocabulary 检测(Grounding-DINO 或 YoloE)

13. 记忆层

模块	存储内容	用途
SemanticMapperModule	SceneGraph → RoomObjectKG + TopologySemGraph,30s 自动存盘	场景级对象知识
EpisodicMemoryModule	机器人经历的”事件”	回溯 / 报告
TaggedLocationsModule	用户命名的位置(“厨房”,“充电桩”)	高优先 goal resolution
VectorMemoryModule	CLIP embedding 向量库	模糊语义检索
TopologicalMemory	拓扑图节点 + 连通关系	frontier 探索

存储路径: `data/memory/global|sites|robots|missions/` (Markdown 是长期真源,向量库只是缓存)。

14. Agent Loop — LLM 多步

`src/semantic/planner/.../agent_loop.py`:观察 → 思考 → 行动循环,7 个 LLM 工具: - `navigate_to(x, y)` - `navigate_to_object(label)` - `detect_object(query)` -

`query_memory(text)` - `tag_location(name)` - `say(text)` — 对话反馈 - `done(summary)` — 终止

限制:最多 10 步 / 120s。支持 OpenAI function-calling + text JSON fallback。

支持的 LLM(`@register("llm", ...)`): | 注册名 | 环境变量 | |—|—| | `kimi` | `MOONSHOT_API_KEY` (国内直连) | | `openai` | `OPENAI_API_KEY` | | `claude` | `ANTHROPIC_API_KEY` | | `qwen` | `DASHSCOPE_API_KEY` (国内备胎) | | `mock` | 不调外部 |

第五部分 · 安全与接口

15. Safety 层

SafetyRingModule(L0): - 输入:obstacle_map, safety_distance, dialogue_state, current path - 输出: `stop_cmd` (0=clear, 1=soft 减速, 2=hard 急停) - 硬停:前向 0.3m 有障碍 + 速度 >0.2 m/s - 软停:前向 0.8m 有障碍 - 对话状态:用户说”继续”,softstop 松开;“停”,升级 hardstop

GeofenceManagerModule: - 读 `config/geofence.yaml` ,圆形 / 多边形边界 - 发 `boundary` 点云给 LocalPlanner 作硬障碍 - 机器人越界:触发 safety stop

急停分发:

```
bp.wire("SafetyRingModule", "stop_cmd", "ThunderDriver", "stop_signal")
bp.wire("SafetyRingModule", "stop_cmd", "NavigationModule", "stop_signal")
```

16. Gateway / Dashboard / MCP

GatewayModule(L6, `src/gateway/gateway_module.py`): - FastAPI + uvicorn,端口 5050 - **HTTP**: `/api/v1/*` 约 30 个 endpoint - **SSE**: `/api/v1/events` 推送 odometry / mission / scene_graph / map_cloud / ... - **WebSocket**: `/ws/teleop` (JPEG 摇杆)、 `/ws/cloud` (二进制点云)、 `/ws/webrtc/*` (视频流) - 静态托管 Dashboard(`web/dist/`)

主要 endpoints: - `GET /api/v1/session` — 当前模式 + 地图 + ICP quality - `POST /api/v1/session/start {mode, map_name}` — 切模式(自动启动 slam / localizer) - `POST /api/v1/session/end` — 退回 idle - `POST /api/v1/goal {x, y, z}` — 发导航目标 - `POST /api/v1/slam/relocalize {map_name, x, y, yaw}` — 手动重定位 - `POST /api/v1/cmd/stop` — 急停 - `POST /api/v1/bag/start {duration, prefix}`

— 录 bag - `POST /api/v1/webrtc/bitrate {bps}` — 动态调视频码率 - `GET /api/v1/webrtc/stats` — 视频流统计 - `POST /api/v1/map_cloud/reset` — 清空浏览器侧累积点云

MCPServerModule(`src/gateway/mcp_server.py`): - JSON-RPC 端口 `8090/mcp` - 16 个 tool(auto-discovered `@skill`)供 Claude / GPT 通过 MCP 协议直接控制 - 注册: `claude mcp add --transport http lingtu http://192.168.66.190:8090/mcp`

重定位位姿持久化(本 session 新加): - 成功 `/slam/relocalize` 后 → `~/.lingtu/last_nav_pose.json` 原子落盘 - 下次 `session/start navigating` 后台 worker 2.5s 后自动调 relocalize 用持久化位姿 - 意义:断电重启不用再 shift+click

17. Teleop + 录 bag

TeleopModule(`src/drivers/teleop_module.py`): - 订阅 `color_image` ,JPEG 编码 30 fps,推 `/ws/teleop` - 接收浏览器 WebSocket 摇杆: `{"type":"joy", "lx":0.5, "ly":0.0, "az":-0.3}` - 内置 3s 空闲自动松手 - 发 cmd_vel → CmdVelMux(优先级 100)

录 bag(`scripts/record_bag.sh`): - `ros2 bag record` 13 个话题(nav / exploration / localization / camera_info) - 不录相机 raw(避免数据爆炸) - 存 `~/data/bags/<prefix>_<ts>/`

录制话题清单:

```
/nav/{lidar_scan, imu, odometry, map_cloud, registered_cloud, cmd_vel, goal_pose}
/localization_quality
/exploration/{way_point, path, runtime, finish}
/camera/camera_info
```

视频流方案对比

方案	延迟	码率	CPU	状态
JPEG-over-WS(`/ws/teleop`)	200 ms	4 Mbps	30% 一核	生产可用
WebRTC + H.264(本 session 尝试)	100 ms 目标	1.5 Mbps	30% 一核	码率调节 API 通 · 编码器卡住(aarch64 libx264)

方案	延迟	码率	CPU	状态
Phase B: hobot_codec BPU 硬编	<50 ms	1.5 Mbps	<5%	**路线图**

当前森哥用 JPEG 足够,WebRTC 等硬编方案。

第六部分 · 运维与外场

18. S100P 部署

目录布局:

```
~/data/SLAM/navigation/          → ~/data/inovxio/lingtu/ (软链接)
~/data/inovxio/lingtu/           源码根
  ├── lingtu.py                  CLI 入口
  ├── cli/profiles_data.py       profile 配置
  ├── src/                       Python 源码
  ├── install/                   colcon 构建(ROS2 package)
  └── web/dist/                  Dashboard 静态资源
/tmp/lingtu_nav.log              tmux 内 lingtu 运行日志
~/data/inovxio/data/maps/        建图输出
  └── <map_name>/
        ├── map.pcd              SLAM 点云
        ├── tomogram.pickle      PCT 输入
        ├── map.pgm + map.yaml   RVIZ 兼容
        └── poses.txt            PGO 输出
~/data/bags/                     rosbag 存放
~/.lingtu/last_nav_pose.json     断电恢复持久化位姿
/etc/systemd/system/             slam.service / localizer.service / camera.service
/usr/local/bin/lt                外场 CLI 脚本
/opt/ros/humble/                 ROS2 Humble
```

systemd 服务: - `slam.service` — Fast-LIO2 + Livox driver - `localizer.service` — ICP Localizer(加载 saved map) - `camera.service` — Orbbec launch - `lingtu.service` — *当前未用*,外场用 tmux 手动启动

启动方式(tmux 稳定模式):

```
tmux kill-session -t lingtu 2>/dev/null
tmux new-session -d -s lingtu -x 200 -y 50 \
  'cd ~/data/SLAM/navigation && python3 lingtu.py nav --llm mock --no-repl 2>&1 |
        tee /tmp/lingtu_nav.log'
```

防火墙: `iptables ROBOT_REMOTE` 默认 DROP,需显式开放 5050/8090。 `lt restart` 后检查。

CycloneDDS:生产用 raw cyclonedds(不走 rclpy),S100P 已验证。本地 Windows 用 DDS fallback 需额外配。

19. 外场操作

5 秒健康检查

```
ssh sunrise@192.168.66.190
lt
```

输出关键字段 → 判断表:

指标	绿	黄	红
mode	navigating/mapping	idle	空
icp_quality	0 < x < 0.3	0.3-0.5	0 / -1
localizer_ready	true	—	false
/nav/odometry	8-10 Hz	3-8	<3 / 空
/localization_quality	~10 Hz	—	空

红了 → 第 20 节故障处理。

导航流程(90% 外场任务)

```
# Step 1: 确认在导航模式
lt                              # 看 mode
lt session nav                  # 若 idle → 切到导航(默认加载上次用的地图)
lt session nav my_custom_map    # 指定地图

# Step 2: 重定位(给 ICP 一个合理初始位姿)
lt reloc 0 0 0                  # 如果机器人在地图原点朝东
lt reloc 2.5 -1.3 1.57          # 如果在(2.5, -1.3)朝北
#  → 等 3 秒看 icp_quality < 0.3 为成功
```

```
# Step 3: 发导航目标
lt nav 5 3                    # 去(5, 3)

# Step 4: 看进度 / 急停
lt                            # 看 mission 状态
lt stop                       # 急停,cmd_vel 归零

# Step 5: 录 bag(故障复现)
lt record start               # 开始
lt record                     # 查状态
lt record stop                # 结束
```

建图流程

```
# 切到建图模式
lt session end
lt session map

# 遥控或自主走完场地
# ...

# 保存地图(触发完整 pipeline:pcd + tomogram + occupancy)
lt map save my_lab_20260419

# 切到导航用新地图
lt session end
lt map use my_lab_20260419
lt session nav my_lab_20260419
```

避障(自动)

避障**不需要你主动开**,只要: 1. Session 在 `navigating` 模式 2. `icp_quality` 正常 3. 发了导航目标

系统自动: - LocalPlanner 每帧重打分 1000 条候选路径 - Terrain 输出可通行性 - CmdVelMux 按优先级选最终速度

20. 故障处理速查表

症状	判断	修复
浏览器打不开 Dashboard	`ping 192.168.66.190` 不通	检查 WiFi / 机器人电源
`lt` 所有命令 “connection refused”	`ps -ef | grep lingtu` 无结果	`lt restart`
`icp_quality = -1` 或 `0`	ICP 从(0,0,0)对不上	`lt reloc X Y YAW`(估计真实位置)
位置显示 `(-5000, 20000)` 之类	Fast-LIO2 静置漂移	`sudo systemctl restart slam` 然后 `lt reloc`
浏览器摄像头黑屏	`lt cam` 重启;若 `lsusb | grep Bootloader`	**物理拔插 Orbbec USB**
规划失败	`lt log` 看错误(目标在障碍里?)	换目标点
点云和 saved map 错位	localizer 没 ready	先 reloc
`lt` 命令不存在	未装	`scp lt → /usr/local/bin/lt + chmod +x`

踩过的坑(已记录): - **Orbbec `color_fps:=30` 锁 bootloader** — 不要给 `gemini_330_series.launch.py` 传这参数,否则固件卡死要物理拔插 - **rclpy + uvicorn + aiortc 启动冲突** — 三者 thread 调度不兼容,用 DDS fallback 绕开 - **iptables ROBOT_REMOTE 默认挡 5050/8090** — 每次部署或重启要开 - **SSH 长命令含分号 exit 255** — 写 heredoc 脚本再执行

21. 标定

出厂标定工具箱: `calibration/` ,覆盖 S100P 全部传感器。

SOP: | 步骤 | 内容 | 工具 | 时间 | |—|—|—|—| | 1 | 相机内参(棋盘格 9×6) | `calibration/camera/calibrate_intrinsic.py` | ~5 min | | 2 | IMU 噪声(Allan Variance) | `calibration/imu/allan_variance_ros2/` | 2-3 hr | | 3 | LiDAR-IMU 外参(8 字运动) | `calibration/lidar_imu/LiDAR_IMU_Init/` | ~2 min | | 4 | 相机-LiDAR 外参(target-less) | `calibration/camera_lidar/direct_visual_lidar_calibration/` | ~10 min | | 5 | 一键应用 | `calibration/apply_calibration.py` | 秒 | | 6 | 一键验证 | `calibration/verify.py` | 秒 |

标定参数归宿: `config/robot_config.yaml` (single source of truth)。

运行时校验: `src/core/utils/calibration_check.py` 启动时检查,FAIL 级阻止启动(焦距为 0、旋转矩阵非正交)。

第七部分 · 附录

22. 性能指标(S100P aarch64)

阶段	耗时
PCT 全局规划(30m 场地)	80-300 ms
LocalPlanner 一帧(1000 path × 36 rot)	3-8 ms
PathFollower 一帧	<1 ms
CmdVelMux 仲裁	<0.1 ms
Fast Path goal resolve	0.17 ms
Slow Path(LLM)goal resolve	~2 s
JPEG 编码(720p)	30-35 ms
CLIP 编码 1 帧	45 ms(BPU)/ 150 ms(CPU)
端到端 goal → cmd_vel(稳态)	**<15 ms**
端到端 goal → 机器人动(冷启 PCT)	**~300 ms**

23. 已知限制 / 路线图

红色(生产阻塞): | # | 问题 | 状态 | |—|—|—| | R1 | Fast-LIO2 静置 >1h 漂移爆表 | **修复中** (C++ IEKF 协方差上限) | | R2 | WebRTC libx264 在 aarch64 encoder 卡住 | **路线图** (hobot_codec BPU 硬编) |

黄色(外场体验): | # | 问题 | 状态 | |—|—|—| | Y1 | 相机默认 7 fps(orbbec color_fps=0 自选) | 参数组合未找到稳定 30 fps 方案 | | Y2 | PCT 仅 aarch64,x86 无法复现 | 接受,A* 备胎 | | Y3 | ChromaDB 可选,缺失时 VectorMemory 软失败 | 接受 | | Y4 | 真实 ROS2 集成测试缺失,1537 pytest 全 mock | 路线图 |

绿色(优化): | # | 问题 | 状态 | |—|—|—| | G1 | gateway_module.py 2700+ 行,耦合严重 | 路线图,Session + SSE + HTTP 拆开 | | G2 | Monitoring / alerting 缺失 | 路线图,Prometheus + 企业微信 |

24. CLI / HTTP API / FAQ

`lt` **命令**(SSH 后用):

```
lt                      一眼看全系统
lt nav X Y              发导航目标
lt reloc X Y YAW        重定位(弧度 yaw)
lt stop                 急停

lt session nav [MAP]    切导航模式
lt session map          切建图模式
lt session end          结束 session

lt map list             列地图
lt map save <name>      保存
lt map use <name>       切换

lt record start/stop    录 bag
lt log / lt logf        看日志
lt restart              重启 lingtu
lt cam                  重启相机
```

关键 HTTP 路径: `http://192.168.66.190:5050/api/v1/` 见第 16 节 Gateway 清单。

关键文件路径速查:

```
~/data/SLAM/navigation/              lingtu 源码(软链)
/tmp/lingtu_nav.log                  运行日志
~/data/inovxio/data/maps/            地图
~/data/bags/                         rosbag
~/.lingtu/last_nav_pose.json         重定位持久化
/etc/systemd/system/{slam,localizer,camera}.service
/usr/local/bin/lt                    外场 CLI
config/robot_config.yaml             机器人参数(标定 + v_target + lookahead 等所有)
cli/profiles_data.py                 profile 配置
```

FAQ:

Q: 为什么不直接用 nav2? A: nav2 太重,四足不需要 behavior tree 那一套;PCT 的 3D 地形规划 nav2 没有;我们要 Module-First 不走 lifecycle node。但 OccupancyGrid 兼容 nav2

map server(map.pgm + map.yaml 输出),可以互用地图。

Q: 建图时机器人能不能远程控制? A: 能。建图模式下依然可以 `lt nav X Y` (PCT 规划走已建图区域)或摇杆,但通常手动推 / 摇杆效果更好。

Q: 没网能用吗? A: 能。LLM 用 `--llm mock` 走关键词路径(Fast Path 够用)。只有 Slow Path(多步 reasoning)需要云端 API。

Q: 跑 lingtu.py 本地(Windows)? A: 能(profile `dev` 或 `sim`)。PCT 自动降 A*,CameraBridge stub 模式,SLAM `none`。适合改代码 + 测框架,不适合实测导航。

Q: 断电后机器人丢自己的位置吗? A: 本 session 修了 — `~/.lingtu/last_nav_pose.json` 持久化上次成功重定位的位姿,下次 `session/start navigating` 后台 worker 自动恢复,不用再 shift+click。

Q: 两个操作路径冲突吗? A: Dashboard 和 `lt` 共用同一套后端 API,同时用没问题。浏览器看画面, `lt` 发命令是常见组合。

致谢

- **PCT_Planner**:HKU/HKUST Bowen Yang & Jie Cheng,GPLv2
- **base_autonomy**(LocalPlanner / PathFollower / Terrain):CMU Chao Cao 团队 TARE
- **Fast-LIO2**:HKU-Mars Xu Wei
- **Orbbec SDK**:奥比中光
- **Livox ROS2 driver**:Livox 开源

本手册是 LingTu 的单一真源。子文档(按主题)见 `docs/*` ,操作细节变更请同步更新这里。